Research Intake 2026

Day 30

Advanced CNN Architectures

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

The future of AI is bright, and you can be part of it!

1 The Architecture Evolution Story

This day picks up directly from where Day 29 left off. Day 29 §6 established the classic conv-pool-flatten-dense template — LeNet's original structure, scaled up by AlexNet — and Day 29 §6.5 promised that this day would survey how that basic template evolved into today's much deeper, more sophisticated backbone architectures. Day 29 §1.2 and Day 26 §6.1 already covered the 2012 AlexNet moment itself in full; this day deliberately does not re-tell that story a third time, instead picking up the question AlexNet's success immediately raised: given that deep, learned convolutional features work, how did architectures evolve from AlexNet's original design to today's sophisticated backbones?

It is worth being explicit about the scope and structure of this day before diving into individual architectures, since it covers considerably more ground than a single named model. Sections 2 through 4 trace the "pure depth" lineage — VGG's disciplined simplicity, the vanishing gradient problem that limited how far that lineage could scale unmodified, and ResNet's residual-connection solution. Section 5 covers Inception's alternative, width-and-multi-scale-driven lineage. Section 6 shifts the optimisation target entirely, from maximum accuracy to accuracy-per-unit-of-compute. Section 7 introduces an architecturally distinct alternative to convolution altogether. Section 8 covers the hybrids emerging where these lineages meet. And Section 9 synthesises everything into a single practical decision framework — the section this entire day has been building toward.

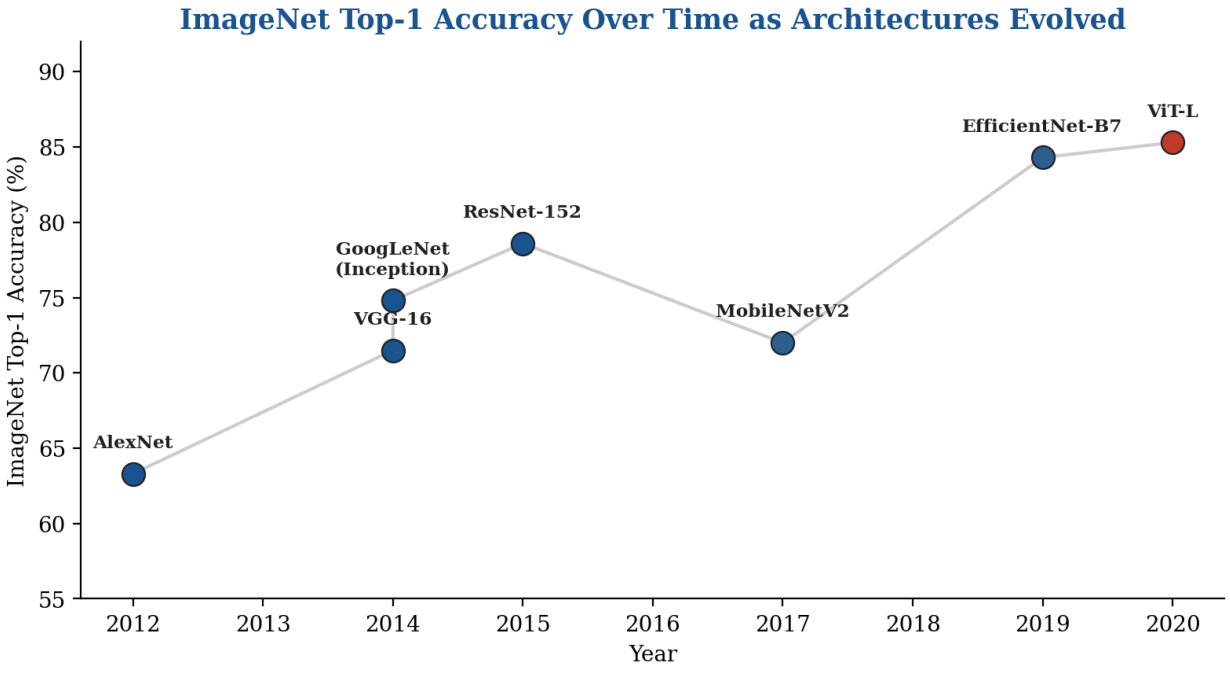

Figure 1. *ImageNet top-1 accuracy over time as architectures evolved from AlexNet through the CNN lineage to Vision Transformers — illustrating the pace of progress this day traces.*

1.1 The Recurring Tension: Depth, Width, and Efficiency

Every architectural innovation this day covers can be understood as a response to one recurring tension: a network needs to be deep and wide enough to represent complex visual patterns, but every additional layer

and every additional channel adds computational cost and, past a certain point, training difficulty. VGG (Section 2) pushed depth using a disciplined, simple design. The vanishing gradient problem (Section 3) revealed that naively stacking more layers eventually hurts rather than helps, motivating ResNet's residual connections (Section 4) as a targeted fix. Inception (Section 5) pursued width and multi-scale processing as an alternative axis of improvement to pure depth. MobileNet and EfficientNet (Section 6) confronted the same tension from the opposite direction, optimising for a fixed, tight computational budget rather than for maximum accuracy. And Vision Transformers (Section 7) represent a genuinely different point in the design space entirely, trading away convolution's built-in efficiency-through-locality for a more flexible but more data-hungry alternative. Keeping this single underlying tension in view — accuracy and representational capacity, weighed against computational and data cost — makes the entire architecture lineage covered in this day considerably easier to follow as one continuous story rather than a list of unrelated named models.

1.2 A Timeline View

Figure 1 lays out this lineage chronologically: AlexNet (2012, Day 26 §6.1 and Day 29 §1.2) → VGG and GoogLeNet/Inception (2014, Sections 2 and 5) → ResNet (2015, Sections 3–4) → MobileNet and EfficientNet (2017–2019, Section 6) → Vision Transformers (2020 onward, Section 7). Two things are worth noting about this timeline before diving into the individual architectures. First, accuracy gains were not smooth or guaranteed — each jump reflects a genuine architectural insight solving a specific, identified problem with what came before, not merely incremental scaling. Second, and less visible from accuracy numbers alone, the field's optimisation target diversified over time: the earliest entries in this timeline optimised almost purely for ImageNet accuracy, while later entries (MobileNet and EfficientNet especially) explicitly optimised for accuracy-per-unit-of-compute, reflecting the field's maturing understanding that "biggest possible network" is the right goal for only a subset of real deployment scenarios — a theme Section 9's backbone-selection framework returns to directly.

It is also worth noting what this timeline does not show: the underlying datasets and evaluation protocols used to measure "ImageNet accuracy" were themselves refined over this same period, meaning the accuracy figures in Figure 1 are broadly, but not perfectly, comparable across the full ten-year span. Later entries in the timeline benefit not only from architectural improvements but also from a decade of accumulated best practice around data augmentation (Day 27 §6), training recipes (Day 29 §7), and regularisation (Day 29 §8) — refinements that, applied retroactively, can noticeably improve even an older architecture's reported accuracy relative to its original published figure. This is a first, concrete instance of the fair-comparison caveat this day's §11.1 develops in full: raw accuracy numbers drawn from different papers, years, and training setups are a useful guide, but should be read with some caution rather than treated as perfectly precise.

Reading the Timeline: Progress Was Neither Smooth Nor Free

Each named architecture in Figure 1 represents a genuine research contribution addressing a specific, previously identified weakness — VGG's systematic simplicity addressed AlexNet's somewhat ad-hoc layer sizing; ResNet's residual connections directly addressed the vanishing-gradient limit on trainable depth (Section 3); EfficientNet's compound scaling addressed the inefficiency of scaling width, depth, or resolution in isolation (Section 6).

None of these gains were free: deeper and wider networks generally cost more to train and run, and the entries positioned toward the timeline's upper-right (higher accuracy) are not automatically the "best" choice for every deployment — Section 9's decision framework exists precisely because the right architecture depends on constraints Figure 1's accuracy-only view does not capture.

Architecture	Year	Primary Innovation	Primary Motivation
AlexNet	2012	Deep CNN + ReLU + dropout at scale	Prove learned features beat handcrafted ones (Day 26 §6)
VGG	2014	Stacked small 3×3 kernels	Simplicity and disciplined depth (§2)
GoogLeNet/Inception	2014	Parallel multi-scale modules	Let the network choose its receptive field (§5)
ResNet	2015	Residual/skip connections	Make extreme depth trainable (§3–§4)
MobileNet	2017	Depthwise separable convolutions	Mobile/edge deployment (§6)
EfficientNet	2019	Compound scaling	Optimal accuracy per unit of compute (§6)
ViT	2020	Patches + self-attention, no convolution	Explore an alternative to convolution entirely (§7)

1.3 How This Day Is Organised

Given the breadth of architectures this day surveys, it is worth previewing the organisational logic explicitly. Sections 2 through 4 form a connected narrative arc, not three independent topics: Section 2 (VGG) establishes the pure-depth approach and its limits; Section 3 (vanishing gradients) diagnoses exactly why that approach breaks down past a certain depth; Section 4 (ResNet) presents the architectural fix. A reader should come away from these three sections with a causal understanding — why VGG-style depth stalls, and precisely what residual connections change about the computation to unstall it — rather than three separate facts to memorise independently. Section 5 (Inception) then steps sideways to a different lever entirely (width and multi-scale processing rather than depth), before Section 6 changes the optimisation objective itself (efficiency rather than raw accuracy). Sections 7 and 8 introduce and then hybridise an architecturally distinct alternative to convolution altogether, and Section 9 closes by synthesising every preceding section into one practical framework. Reading the sections in order, rather than jumping directly to a specific named architecture of interest, is likely to make each individual

architecture's design choices considerably easier to understand, since most of them are motivated as a direct response to a limitation identified in an earlier section.

2 VGG: Depth Through Simplicity

2.1 VGG's Core Insight: Stacked Small Kernels

VGG (Simonyan & Zisserman, Oxford, 2014) made a deceptively simple but influential architectural choice: use only small, 3×3 convolutional kernels (Day 29 §3.3) throughout the entire network, stacked in sequence, rather than the larger kernels (7×7, 11×11) earlier architectures like AlexNet had used in their initial layers. Figure 2 illustrates the key mathematical insight behind this choice: three stacked 3×3 convolutional layers achieve the same effective receptive field (Day 29 §3.3) as a single 7×7 convolutional layer, but with meaningfully fewer parameters and, critically, three non-linear activation functions (Day 29 §5) inserted along the way rather than just one.

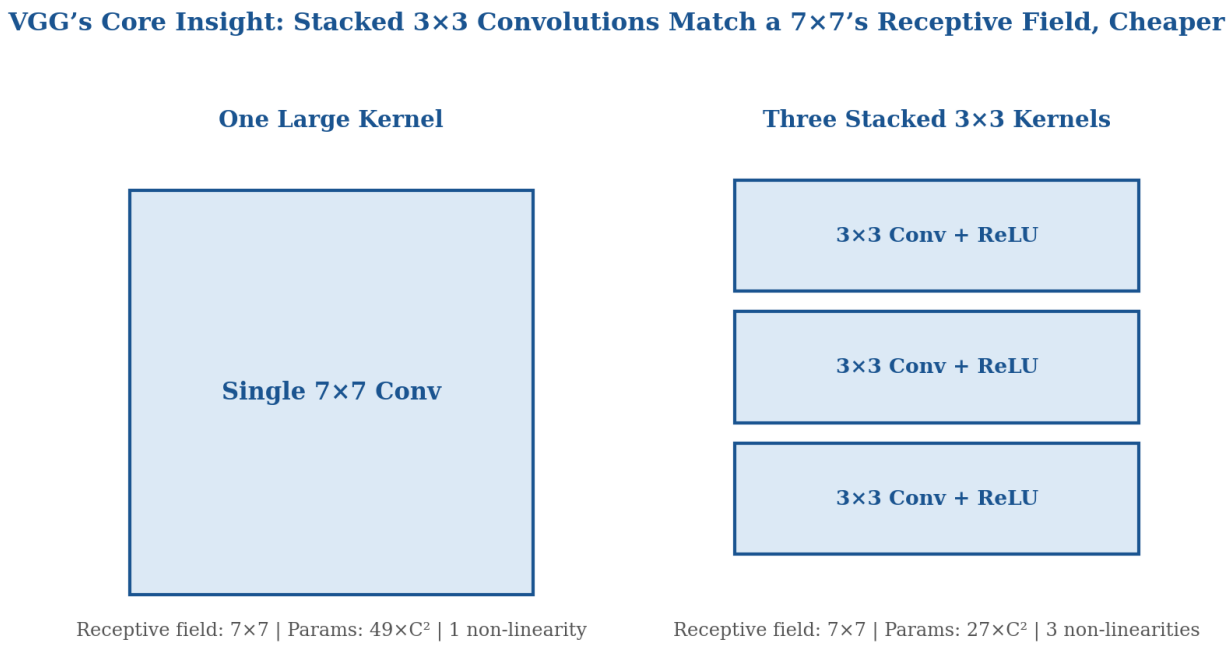

Figure 2. *VGG's core insight: three stacked 3×3 convolutions match a single 7×7 convolution's receptive field, using roughly half the parameters and three non-linearities instead of one.*

Worked Example: Verifying the Parameter Savings

For an input with C channels, a single 7×7 convolutional layer producing C output channels requires $7 \times 7 \times C \times C = 49C^2$ weights.

Three stacked 3×3 layers, each mapping C channels to C channels, require $3 \times (3 \times 3 \times C \times C) = 3 \times 9C^2 = 27C^2$ weights — roughly 45% fewer parameters than the single 7×7 layer, for an identical 7×7 effective receptive field (per Day 29 §3.3's receptive-field-growth formula).

The three extra non-linearities (one after each 3×3 layer, versus just one after the single 7×7 layer) are, if anything, a further advantage rather than a cost — per Day 29 §5.1, more non-linear transformations between the input and output generally increase a network's representational flexibility for a given depth budget.

2.2 VGG-16 and VGG-19

VGG was released in two primary configurations — VGG-16 and VGG-19, named for their 16 and 19 learnable layers respectively — both following the same rigidly consistent design: blocks of stacked 3×3 convolutions (Section 2.1), each block followed by a 2×2 max pooling layer (Day 29 §4.1) that halves spatial resolution, with the number of filters doubling after each pooling stage (64 → 128 → 256 → 512 → 512), directly instantiating the "channel depth increases while spatial dimensions decrease" pattern Day 29 §6.4 identified as the standard architectural template. The network concludes with the classic flatten-and-fully-connected classification head from Day 29 §6.1, essentially unchanged from LeNet and AlexNet's original design.

Worked Example: Tracing VGG-16's Spatial and Channel Dimensions Stage by Stage

Starting from a 224×224×3 input, VGG-16's five convolutional blocks (each ending in a 2×2 max pool, §2.2) transform the representation as follows.

Block 1 (64 filters): 224×224×3 → 224×224×64 → pooled to 112×112×64.

Block 2 (128 filters): 112×112×64 → 112×112×128 → pooled to 56×56×128.

Blocks 3–5 (256, 512, 512 filters): continuing the same pattern down to a final 7×7×512 feature map before the flatten-and-dense classification head (Day 29 §6.1).

This stage-by-stage halving of spatial resolution alongside doubling of channel depth is a direct, concrete instance of Day 29 §6.4's "spatial dimensions shrink, channel depth grows" pattern — VGG follows this template with unusually strict, mechanical regularity compared to AlexNet's less uniform layer sizing, which is itself part of why VGG became such a clear pedagogical example of the pattern.

2.3 Why Simplicity Made VGG Influential

VGG's rigid, uniform design — every convolutional layer using the identical 3×3 kernel size, every pooling layer using the identical 2×2 configuration — made it unusually easy to understand, modify, and reason about compared to AlexNet's more varied layer-by-layer kernel sizing. This simplicity is a large part of why VGG became, for several years, the default choice as a pretrained backbone (Day 26 §6.3, Day 29 §7.5) for transfer learning across a wide range of downstream tasks, despite not being the single most accurate architecture available even at the time of its release — a useful early illustration of Section 9's

later point that "highest published accuracy" and "best practical choice for a given project" are not always the same thing.

This influence extended well beyond image classification, the task VGG was originally designed and evaluated on. Feature maps extracted from a pretrained VGG network became a standard, widely-used ingredient in tasks with no obvious connection to classification at all — perceptual loss functions for image generation and style transfer (a forward reference to Days 34–35's generative modelling content) commonly compare two images by their VGG feature-map similarity rather than their raw pixel similarity, since VGG's intermediate representations were empirically found to capture perceptually meaningful structure that raw pixel comparison misses entirely. This kind of "borrowed as a general-purpose feature extractor, far beyond its original training task" role is a recurring pattern this day will see again with ResNet (Section 4) and, later in this broader research programme, with CLIP's vision encoder.

2.4 VGG's Limitations

VGG's limitations, which directly motivated much of what followed in this day's lineage, stem from the same rigid simplicity that made it influential. VGG-16 alone has roughly 138 million parameters, the overwhelming majority concentrated in its fully-connected classification layers (echoing the parameter-distribution pattern Day 29 §6.4's worked example identified for a much smaller network) — making VGG computationally expensive to train and slow at inference relative to its accuracy, a problem later architectures addressed through Global Average Pooling (Day 29 §4.3) and more parameter-efficient design (Section 6). VGG also does not scale cleanly to much greater depth than its 16–19 layers — attempts to stack substantially more layers using VGG's plain, non-residual design run directly into the vanishing gradient problem this day covers next.

Property	VGG-16	VGG-19
Learnable layers	16	19
Parameters	~138 million	~144 million
Convolutional layers	13	16
Fully-connected layers	3	3
Relative accuracy	Strong baseline	Marginally higher, at added compute cost

2.5 VGG's Legacy in Perceptual Loss Functions

Beyond its historical role as a transfer-learning backbone (Section 2.3), VGG left a second, less obvious legacy directly relevant to Days 34–35's generative modelling content: the "perceptual loss" or "VGG loss" used to train many image generation and style-transfer models. Rather than comparing a generated image to a target image pixel-by-pixel (which correlates poorly with human perceptual judgments of similarity), perceptual loss instead compares the two images' feature-map activations at one or more intermediate VGG layers — the same intermediate representations Section 9's framework and Day 29 §9's filter-visualisation

content discuss in the context of interpretability. Two images that produce similar VGG feature activations tend to look perceptually similar to a human observer, even when their raw pixel values differ substantially — a direct, practical consequence of VGG's intermediate layers having learned a genuinely meaningful hierarchy of visual features (Day 29 §6.4), reusable well beyond the classification task VGG was originally trained on.

3 The Vanishing Gradient Problem & Residual Learning

3.1 The Degradation Problem: Not Just Overfitting

Day 29 §5.2 introduced the vanishing gradient problem conceptually, in the context of why ReLU displaced sigmoid and tanh as the default activation function. This section develops the problem's consequences for network depth specifically, and the solution the field converged on. A naive expectation might be that stacking more layers onto a network like VGG should only ever help, or at worst make no difference — after all, a deeper network could, in principle, simply learn to make its additional layers compute the identity function, leaving performance unchanged. Empirically, this is not what happens: plain (non-residual) networks stacked substantially deeper than VGG's 16–19 layers exhibit a degradation problem — both training and test accuracy get measurably worse with added depth, not merely fail to improve. Critically, this is not simply overfitting (Day 29 §8.1), since overfitting would show improving training accuracy alongside worsening test accuracy; the degradation problem shows both getting worse simultaneously, indicating the network is failing to optimise effectively, not failing to generalise.

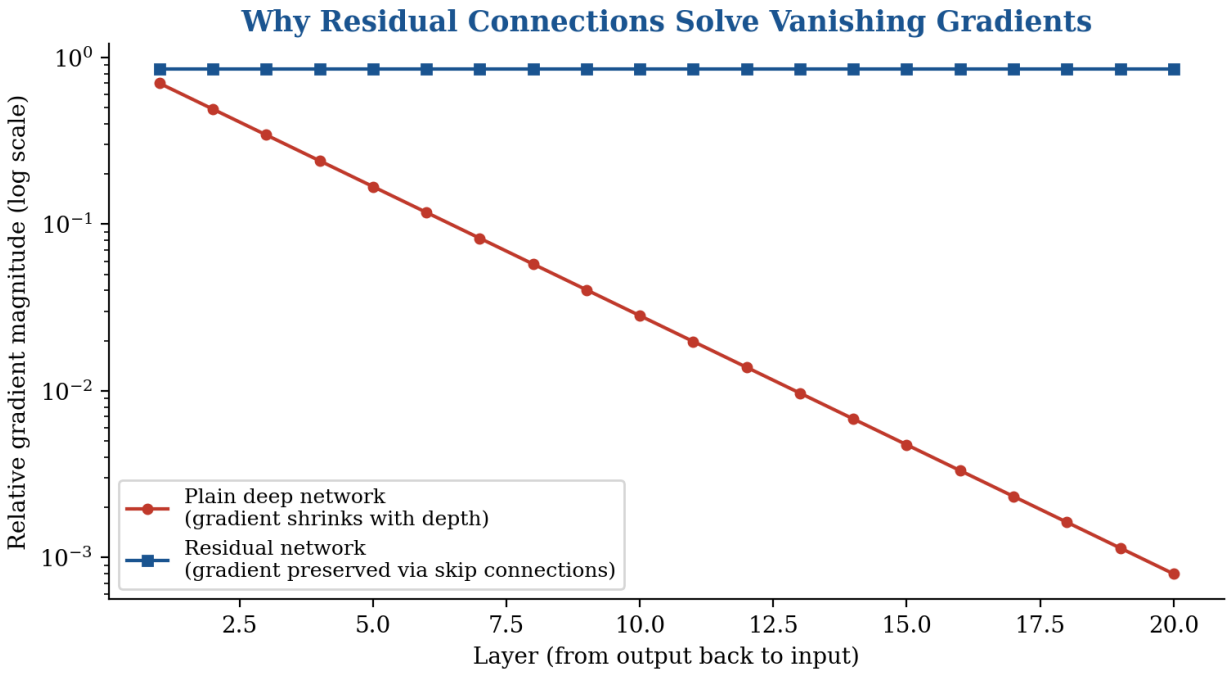

Figure 3. *In a plain deep network, gradient magnitude shrinks rapidly with depth as it backpropagates toward early layers; residual connections keep the gradient signal strong regardless of depth.*

3.2 Why Gradients Vanish with Depth

Figure 3 makes the underlying mechanism concrete: as a gradient backpropagates (Day 29 §7.1) through many stacked layers, it is repeatedly multiplied by each layer's local gradient contribution. Even with ReLU's gradient-preserving behaviour for positive inputs (Day 29 §5.2), the many other multiplicative factors involved in backpropagation through a deep stack of convolutional and normalisation operations tend, empirically, to shrink the accumulated gradient as it travels backward — and this shrinkage compounds multiplicatively with every additional layer, exactly as Day 29 §5.2's sigmoid worked example demonstrated numerically for a much shallower comparison. The practical consequence is that a plain network's earliest layers, in a sufficiently deep stack, receive a gradient signal too weak to train effectively — they remain close to their random initialisation throughout training, unable to learn useful features, which directly explains the degradation problem Section 3.1 described.

Worked Example: Extending Day 29's Gradient Decay Calculation to a VGG-Depth Network

Day 29 §5.2 computed that a per-layer gradient factor of 0.045 (typical of a saturated sigmoid) compounds to roughly 3×10^{-11} after 7 layers.

Even with ReLU's far more favourable per-layer factor, empirically closer to 0.7–0.9 in a typical deep plain network once batch normalization and other practical factors are accounted for, the same multiplicative compounding still applies: $0.8^{20} \approx 0.012$ at VGG-19's 19 layers (Section 2.2) — already a meaningful reduction — but $0.8^{100} \approx 2\times10^{-10}$ at 100 layers, a depth ResNet (Section 4) trains successfully but a plain network of VGG's design cannot.

This is precisely why simply stacking more VGG-style layers does not scale past a certain depth without a structural change to how gradients flow — the compounding multiplicative decay is a property of the plain, non-residual architecture itself, not of any specific layer count.

3.3 The Residual/Skip-Connection Insight

He et al. (Microsoft Research, 2015) proposed a direct architectural solution: rather than asking a stack of layers to learn a desired underlying mapping H(x) directly, restructure the layers to learn a residual mapping F(x) = H(x) − x instead, and recover the desired output by adding the original input back: output = F(x) + x. This "skip connection" or "shortcut connection" — a direct path carrying the input x around the stacked layers, added back to their output — is the single architectural change that defines ResNet, and Section 4 develops its exact implementation in full.

3.4 Why Identity Shortcuts Help Gradient Flow

The intuition for why this reformulation solves the degradation problem is worth making explicit, since it is easy to state the mechanism (Section 3.3) without understanding why it actually works. If the desired mapping H(x) genuinely is close to the identity function for a given block (a very plausible scenario for at least some of the layers in a very deep network, per Section 3.1's "should be able to learn identity" observation), then the residual F(x) = H(x) − x that the stacked layers need to learn is close to zero — a target that is considerably easier for gradient descent to discover and represent than the identity function itself would be through a plain, non-residual stack of non-linear layers. More fundamentally, from a pure

gradient-flow perspective: because the skip connection provides a direct, unimpeded additive path from a block's output back to its input, gradients can flow backward through that shortcut path essentially unchanged, regardless of how poorly-conditioned the gradient flow through the stacked convolutional layers happens to be — exactly the "gradient preserved regardless of depth" behaviour Figure 3's residual curve illustrates. This is why residual networks can be trained successfully at depths (over 100 layers, and eventually over 1,000 in some research settings) where plain networks following VGG's design would suffer severe degradation.

3.5 Empirical Validation: What the Original ResNet Experiment Showed

The original ResNet paper (He et al., 2015) included a direct, carefully controlled experiment worth describing explicitly, since it is the empirical evidence that turned Sections 3.3–3.4's theoretical argument into an accepted, field-changing result rather than a plausible-sounding hypothesis. The authors trained a plain 34-layer network and a residual 34-layer network — identical in every respect except for the presence of skip connections — alongside their shallower 18-layer counterparts, on the same data with the same training recipe. The plain 34-layer network performed worse than the plain 18-layer network on both training and validation accuracy, directly reproducing the degradation problem (Section 3.1). The residual 34-layer network, by contrast, outperformed the residual 18-layer network, exactly as a naive "more capacity should help" intuition would predict — and outperformed the plain 34-layer network by a wide margin despite having identical depth and nearly identical parameter count, isolating the skip connections themselves as the specific cause of the improvement rather than any other confounding factor. This controlled comparison — same depth, same data, same training recipe, skip connections as the only variable — is what made the residual-connection explanation convincing rather than merely plausible, and it is a useful methodological model for Section 11.1's fair-comparison guidance more broadly: isolate one variable at a time when attributing an accuracy difference to a specific architectural change.

3.6 A Note on Highway Networks: A Related, Earlier Idea

It is worth briefly acknowledging Highway Networks (Srivastava et al., 2015), published shortly before ResNet, since they proposed a closely related solution to the same degradation problem (Section 3.1) using a gating mechanism rather than ResNet's simple, parameter-free additive skip connection. A Highway Network layer learns a "transform gate" that decides, for each unit, how much of the input should pass through unchanged (via a learned gate value) versus how much should be replaced by the layer's transformation — a more flexible but also more complex mechanism than ResNet's fixed, always-on identity shortcut. ResNet's simpler design proved both easier to train reliably and, empirically, at least as effective, which is a large part of why the residual-connection formulation, rather than Highway Networks' gating mechanism, became the standard approach adopted across the rest of this day's architecture lineage — a useful reminder that the simplest mechanism solving a given problem often wins out over a more flexible but more complex alternative, all else being equal.

4 ResNet & Its Variants

4.1 Residual Block Anatomy

Figure 4 shows the anatomy of a bottleneck residual block, the specific design used in ResNet's deeper variants (50 layers and beyond). A 1×1 convolution (Day 29 §3.4) first reduces the channel dimension, a 3×3 convolution then operates on this reduced, cheaper representation, and a final 1×1 convolution restores the channel dimension back to its original size — a "bottleneck" shape that keeps the expensive 3×3 spatial convolution's computational cost low, since it operates on fewer channels than the block's input and output. Batch normalization (Day 29 §8.3) follows each convolution, and the skip connection (Section 3.3) adds the block's original input directly to the output of this three-convolution stack, before a final ReLU activation (Day 29 §5.2).

Worked Example: Bottleneck vs. Plain Block: A Cost Comparison

Consider a residual block processing 256 input and output channels, with a bottleneck reduction to 64 channels.

Plain block (two 3×3 convs at full 256 channels): 2 × (3×3×256×256) = 1,179,648 weights.

Bottleneck block (1×1 reduce, 3×3 process, 1×1 restore): (1×1×256×64) + (3×3×64×64) + (1×1×64×256) = 16,384 + 36,864 + 16,384 = 69,632 weights.

The bottleneck design uses roughly 17× fewer parameters than the plain equivalent at the same input/output channel count — the direct reason bottleneck blocks (§4.2) are used specifically in ResNet's deeper, wider-channel variants, where a plain block's cost would otherwise become prohibitive.

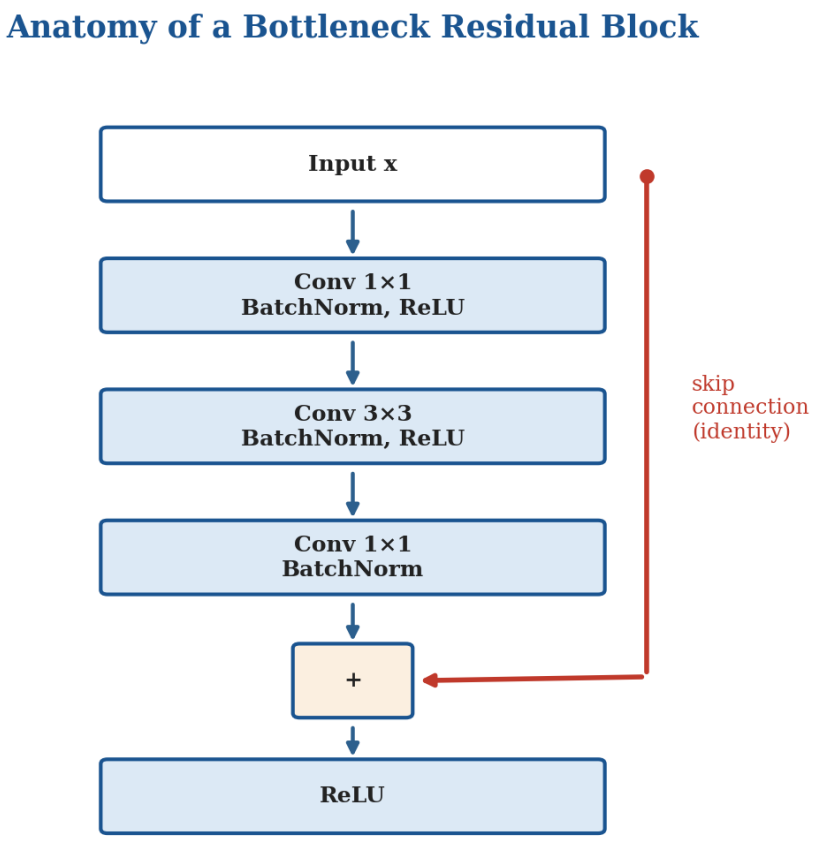

Figure 4. *The anatomy of a bottleneck residual block: a 1×1-3×3-1×1 convolution stack with batch normalization, plus a skip connection carrying the original input around the stack to be added back at the end.*

4.2 The Basic Block vs. the Bottleneck Block

ResNet actually uses two distinct block designs depending on network depth. The basic block, used in the shallower ResNet-18 and ResNet-34 variants, consists simply of two stacked 3×3 convolutions with a skip connection around them — no channel-reduction bottleneck. The bottleneck block, shown in Figure 4 and used in the deeper ResNet-50, ResNet-101, and ResNet-152 variants, adds the 1×1-reduce, 3×3-process, 1×1-restore structure specifically to keep computational cost manageable as the network's overall channel width grows — without the bottleneck design, a plain two-convolution block at the wider channel counts used in these deeper variants would be prohibitively expensive.

4.3 The ResNet Family: 18/34/50/101/152

ResNet was released in several depths, allowing practitioners to directly trade off accuracy against computational cost by choosing a specific variant — illustrated in Figure 5, which shows depth increasing alongside (though with diminishing returns relative to) accuracy across the family.

This diminishing-returns pattern deserves a moment's direct attention, since it is easy to look at Figure 5 and conclude simply "bigger is better, choose the largest variant your compute allows." The accuracy gain from ResNet-18 to ResNet-50 is substantial (roughly 6 percentage points); the gain from ResNet-50 to ResNet-152, despite representing a 3× increase in depth, is considerably smaller (roughly 2 percentage points). This pattern — steep initial gains from added capacity, followed by rapidly diminishing returns — recurs across essentially every architecture family this day covers, and is the direct empirical justification for Section 9's point that ResNet-50 specifically, rather than one of its larger siblings, has become the field's most common default: it sits close to the "knee" of this diminishing-returns curve, capturing most of the family's achievable accuracy at a fraction of ResNet-152's computational cost.

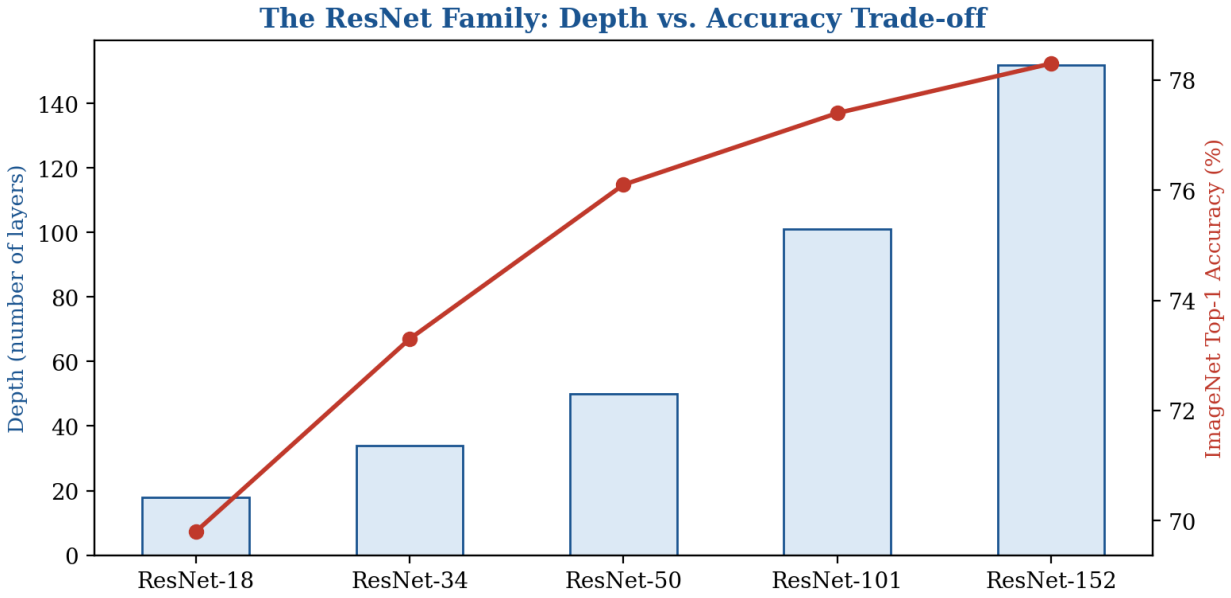

Figure 5. *The ResNet family spans a range of depths, with accuracy improving as depth increases, though with clearly diminishing returns beyond ResNet-50.*

Variant	Block Type	Depth	Typical Use Case
ResNet-18	Basic	18 layers	Fast inference; resource-constrained settings
ResNet-34	Basic	34 layers	A modest step up in accuracy over ResNet-18
ResNet-50	Bottleneck	50 layers	The most common default pretrained backbone
ResNet-101	Bottleneck	101 layers	Higher accuracy where compute allows
ResNet-152	Bottleneck	152 layers	Near-maximum accuracy within the family

ResNet's Lasting Influence

ResNet-50, released in 2015, remains one of the most widely used pretrained backbones in computer vision practice a decade later — not because it is the most accurate architecture available (Section 6 and Section 7's later entries generally exceed it), but because its combination of reasonable accuracy, well-understood behaviour, wide framework support, and abundant pretrained checkpoints makes it a dependable, low-risk default choice for a huge range of downstream tasks.

This longevity is a useful data point for Section 9's decision framework: "well-established and widely supported" is itself a genuine practical advantage, distinct from raw accuracy, and worth weighing explicitly rather than always defaulting to whatever architecture currently tops a benchmark leaderboard.

4.4 Later Refinements: ResNeXt, Wide ResNet, and DenseNet

Several later architectures refined ResNet's residual-connection insight along different axes. ResNeXt introduces "cardinality" as a new dimension alongside depth and width — splitting each residual block's transformation into many parallel, smaller branches (conceptually reminiscent of Inception's multi-branch structure, Section 5) rather than one wide branch, which empirically improves accuracy at a fixed parameter budget. Wide ResNet takes the opposite approach to scaling: rather than adding depth, it increases each layer's width (channel count), finding that a shallower but wider residual network can match or exceed a much deeper, narrower one's accuracy while training faster in wall-clock time, since wide, shallow networks parallelise more efficiently on GPU hardware than narrow, deep ones. DenseNet takes the skip-connection idea further still: rather than each block connecting only to the block immediately before it, every layer in a DenseNet block connects directly to every other layer within that block via concatenation (not addition, unlike ResNet's residual sum) — a dense connectivity pattern that maximises gradient flow and feature reuse throughout the block, addressing the same underlying gradient-flow problem Section 3 described, via a structurally different mechanism than ResNet's simple additive shortcut.

Variant	New Dimension Introduced	Skip-Connection Style	Key Benefit
ResNeXt	Cardinality (parallel branches)	Additive (like ResNet)	Better accuracy at fixed parameter budget
Wide ResNet	Width over depth	Additive (like ResNet)	Faster training via GPU-friendly parallelism
DenseNet	Dense inter-layer connectivity	Concatenation, not addition	Maximal gradient flow and feature reuse

5 Inception & Multi-Scale Feature Extraction

5.1 Inception's Core Insight: Let the Network Choose Its Receptive Field

GoogLeNet (Szegedy et al., Google, 2014), built from repeated Inception modules, pursued a different axis of architectural improvement from VGG's pure-depth approach: rather than committing to a single kernel size at each layer, an Inception module runs several convolutions of different kernel sizes in parallel on the same input, and concatenates their outputs together — illustrated in Figure 6. The motivating insight is that the "right" spatial scale at which to detect a useful feature is not known in advance and likely varies across different parts of an image (a small, fine-grained texture and a large, spatially extended object both need to be detected by the same network, potentially at the same layer), so rather than a human designer committing to one kernel size per layer, the Inception module lets the network learn, via the relative weighting the subsequent layer assigns to each branch's output, which scales matter most for the task at hand.

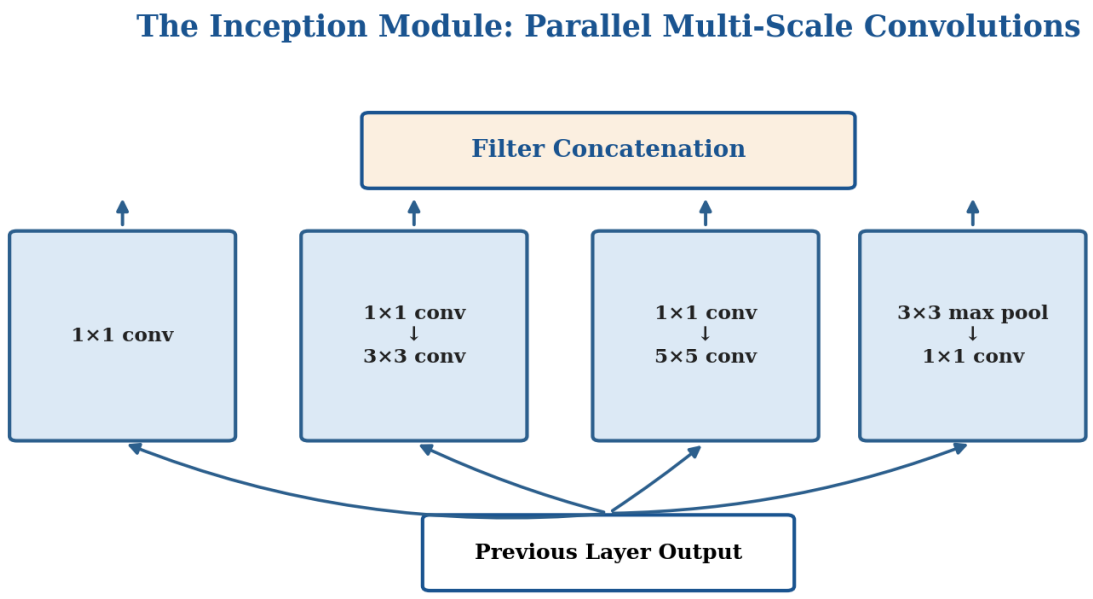

Figure 6. *The Inception module: parallel convolutions at multiple kernel sizes (plus a pooling branch) process the same input simultaneously, with their outputs concatenated — letting the network draw on multiple spatial scales at once.*

5.2 The Role of 1×1 Convolutions in Inception

Notice, in Figure 6, that the 3×3 and 5×5 branches are each preceded by a 1×1 convolution. This is a deliberate, load-bearing design choice, not an incidental detail: per Day 29 §3.4, a 1×1 convolution can reduce channel depth cheaply before a more expensive spatially-larger convolution is applied, dramatically reducing the 5×5 branch's computational cost in particular — without this 1×1 "bottleneck" reduction, a 5×5 convolution operating directly on a wide, many-channel input would be prohibitively expensive to run in parallel alongside the module's other branches. This is the same bottleneck principle Section 4.1's ResNet block also relies on, discovered and applied independently in each architecture for a similar underlying computational-efficiency reason.

Worked Example: Filter Concatenation: Tracking Channel Counts Through an Inception Module

Suppose an Inception module receives a 256-channel input, and its four branches (Figure 6) are configured to output 64, 128, 32, and 32 channels respectively.

After filter concatenation: 64 + 128 + 32 + 32 = 256 output channels — the concatenation simply stacks each branch's output channels together along the channel dimension, rather than summing them the way a residual connection (§3.3) would.

This is a structurally different combination mechanism from ResNet's additive skip connection: Inception's branches must independently be designed (or learned) to be useful once concatenated side-by-side, while ResNet's branches must produce an output compatible in shape for direct element-wise addition — two different design philosophies for how to combine multiple computational paths within a single block.

5.3 GoogLeNet and Factorised Convolutions

GoogLeNet, the original network built from stacked Inception modules, achieved strong ImageNet accuracy (Figure 1) with substantially fewer parameters than VGG, despite being deeper — a direct benefit of Section 5.2's aggressive use of 1×1 bottleneck reductions throughout, combined with GoogLeNet's use of Global Average Pooling (Day 29 §4.3, itself introduced around this time) rather than VGG's large, parameter-heavy fully-connected classification head. Later Inception versions (Inception-v2 and v3) refined the module further by factorising larger convolutions into smaller ones — directly echoing VGG's Section 2.1 insight that a 5×5 convolution can be replaced by two stacked 3×3 convolutions at reduced parameter cost, and going further still by factorising a K×K convolution into a sequence of a 1×K and a K×1 convolution, an even cheaper approximation for larger kernel sizes.

Worked Example: Factorised Convolution: A Parameter Comparison

Consider replacing a 5×5 convolution (mapping C channels to C channels) with a factorised 1×5-then-5×1 sequence.

Standard 5×5: $5\times5\times C\times C = 25C^2$ weights.

Factorised (1×5 then 5×1): $(1\times5\times C\times C) + (5\times1\times C\times C) = 5C^2 + 5C^2 = 10C^2$ weights — a 60% reduction, for an identical 5×5 effective receptive field.

This is a further refinement of the same underlying principle VGG (§2.1) and the standard Inception module (§5.1–§5.2) both already exploit — decomposing a spatially larger operation into a sequence of cheaper, smaller ones without sacrificing the effective receptive field the larger operation provided.

5.4 Inception vs. ResNet: Two Different Philosophies

It is worth contrasting Inception's and ResNet's underlying philosophies directly, since they represent genuinely different answers to Section 1.1's depth-width-efficiency tension. ResNet pursues depth as its primary lever, using residual connections specifically to make extreme depth trainable at all (Section 3). Inception pursues width and multi-scale parallelism as its primary lever, using 1×1 bottlenecks specifically to make that parallel width computationally affordable. Both approaches proved successful, and — as

Section 8 covers — later research has increasingly combined insights from both lineages (residual connections added to Inception-style modules, for instance) rather than treating them as strictly competing alternatives.

Inception-ResNet, released shortly after both original architectures, is worth naming as a direct, concrete illustration of exactly this combination: it replaces each Inception module's filter-concatenation output stage (Section 5.1) with a residual addition instead, adding a skip connection around each Inception module in the style of Section 3.3's ResNet block. The result trains faster than a comparably-sized plain Inception network, directly demonstrating that Section 3's gradient-flow benefits and Section 5's multi-scale benefits are not mutually exclusive design choices, but can be combined within a single architecture to capture advantages from both lineages simultaneously — a pattern of "combine the previous generation's best ideas" that recurs again with Section 8's CNN-transformer hybrids.

Research Spotlight: Neural Architecture Search

Both VGG's stacking pattern and Inception's multi-branch module were designed by human researchers through manual experimentation and intuition — a slow, expertise-intensive process.

Neural Architecture Search (NAS) automates this design process, using a search algorithm (reinforcement learning, evolutionary search, or gradient-based methods) to explore a large space of possible architectural configurations and automatically discover high-performing designs — EfficientNet's baseline architecture (Section 6.3) was itself discovered via NAS, illustrating that architecture design has itself become, in part, a learned rather than a purely hand-engineered process, echoing this module's broader hand-designed-to-learned theme from Day 29 §1.

5.5 Xception: Taking Inception's Logic to Its Extreme

Xception ("Extreme Inception," Chollet, 2016) is worth naming as a direct conceptual bridge between this section's Inception lineage and Section 6's efficiency-focused architectures. Xception starts from the observation that Inception's core operation — a 1×1 convolution followed by a spatially larger convolution on the reduced channel representation (Section 5.2) — is structurally very close to a depthwise separable convolution (the same operation Section 6.2 introduces as MobileNet's central technique), differing mainly in the order of operations and whether a non-linearity is inserted between the two steps. Xception takes this observation to its logical extreme: replacing Inception's multi-branch modules entirely with a deep stack of depthwise separable convolutions, arguing that this "extreme" version of Inception's underlying logic — full separation of cross-channel and spatial correlations, rather than Inception's partial separation via parallel branches — produces a more efficient use of parameters at comparable accuracy. This makes Xception a useful conceptual stepping stone for understanding why Section 6's efficient architectures and this section's multi-scale Inception lineage are more closely related than their different motivating stories (accuracy via multi-scale processing vs. efficiency via factorisation) might initially suggest — both ultimately converge on similar underlying convolutional factorisation techniques.

6 Efficient Architectures for Constrained Deployment

6.1 Motivation: Mobile and Edge Deployment

Every architecture covered so far in this day was designed primarily to maximise ImageNet accuracy, with computational cost a secondary consideration. This section covers a different design objective entirely: architectures optimised specifically for deployment under a tight, fixed computational budget — mobile phones, embedded devices, and real-time applications with strict latency requirements. This is directly relevant to this module's own PP12 face-mask-detection scenario from Day 26, previewed there as a real example of needing a lightweight backbone: a face-mask detector intended to run on an entry-screening camera's on-device hardware, rather than a well-resourced cloud server, needs exactly the kind of architecture this section develops, not the maximum-accuracy-regardless-of-cost architectures covered in Sections 2 through 5.

It is worth being precise about what "computational budget" actually means in practice, since it bundles together several distinct, only loosely correlated constraints. Parameter count determines a model's storage and memory footprint — relevant for devices with limited RAM or storage. FLOPs (floating-point operations) determine the raw computational work required for one forward pass — a reasonable proxy for energy consumption and, on some hardware, latency. Actual measured latency on specific target hardware is the metric that ultimately matters for a real deployment, and — as this day's §11 pitfalls checklist flags explicitly — does not always track FLOPs or parameter count cleanly, since different operations (a 1×1 convolution versus a depthwise convolution of similar FLOP cost, for instance) can have very different real-world latency on a given piece of hardware due to memory-access patterns and hardware-specific optimisation support.

It is worth being explicit about a point Day 26 §9's ethics discussion raises indirectly here: efficient, on-device architectures like the ones this section covers are not purely a matter of engineering convenience. Processing sensitive data (a face, per this module's PP12 scenario; a medical image, per Day 37) entirely on-device, rather than transmitting it to a cloud server for a larger model to process, can be a meaningful privacy advantage — the raw data never leaves the user's device at all. This privacy consideration is a genuine, independent reason to favour an efficient architecture even in situations where server-side compute is technically available and cost is not the binding constraint, and it is worth adding explicitly to Section 9's decision framework as a factor beyond pure latency and accuracy.

6.2 Depthwise Separable Convolutions

MobileNet's central architectural technique, depthwise separable convolution, was already introduced with a full worked cost comparison in Day 29 §10.3 as a preview of this day's content — a standard multi-channel convolution (Day 29 §3.4) is factorised into two cheaper sequential stages: a depthwise stage, applying a single small spatial kernel independently to each input channel with no cross-channel mixing at all, followed by a pointwise stage, a 1×1 convolution (Day 29 §3.4) that performs the cross-channel mixing separately. Day 29's worked example computed roughly an 8× reduction in computational cost from this factorisation alone, for a representative layer configuration — the single largest contributor to MobileNet's efficiency advantage over a standard-convolution architecture of comparable depth.

Use Case: On-Device Real-Time Detection

A retail inventory-scanning app running entirely on a mid-range smartphone, with no cloud connectivity required, uses a MobileNetV3 backbone to classify shelf products in real time at over 30 frames per second — a task a ResNet-50-based pipeline (Section 4) could not achieve on the same hardware within an acceptable latency budget.

This is a direct, deployed illustration of Section 6.1's motivating scenario, and of exactly the trade-off Section 6.4's table formalises: MobileNet trades some raw accuracy, relative to a much larger ResNet or EfficientNet variant, for the ability to run at all under this specific deployment's hard computational constraints.

6.3 EfficientNet and Compound Scaling

EfficientNet (Tan & Le, Google, 2019) addressed a different, complementary question: given a fixed increase in available compute budget, what is the most effective way to scale up a baseline architecture — deeper, wider, or with higher input resolution? Prior practice typically scaled just one of these three dimensions in isolation (VGG-style depth scaling, Wide ResNet-style width scaling, or simply feeding a network larger images). EfficientNet's key finding, illustrated in Figure 7, is that scaling all three dimensions together, in a fixed, principled ratio, produces substantially better accuracy for a given compute budget than scaling any single dimension alone — a technique the paper calls compound scaling, discovered through a small grid search over scaling ratios and then applied uniformly across a family of increasingly large models (EfficientNet-B0 through B7).

The EfficientNet-B0 baseline architecture itself is worth noting as a direct product of Section 5.4's callout on Neural Architecture Search: rather than a human-designed starting point in the style of VGG or the original ResNet, B0 was discovered via NAS, specifically optimised for accuracy-per-FLOP rather than raw accuracy alone — and it is only after this NAS-discovered baseline was established that the compound scaling technique was applied to derive the larger B1 through B7 variants from it. This two-stage process — NAS to find an efficient baseline, then principled scaling to grow it — is itself a template that later efficient-architecture research has continued to build on.

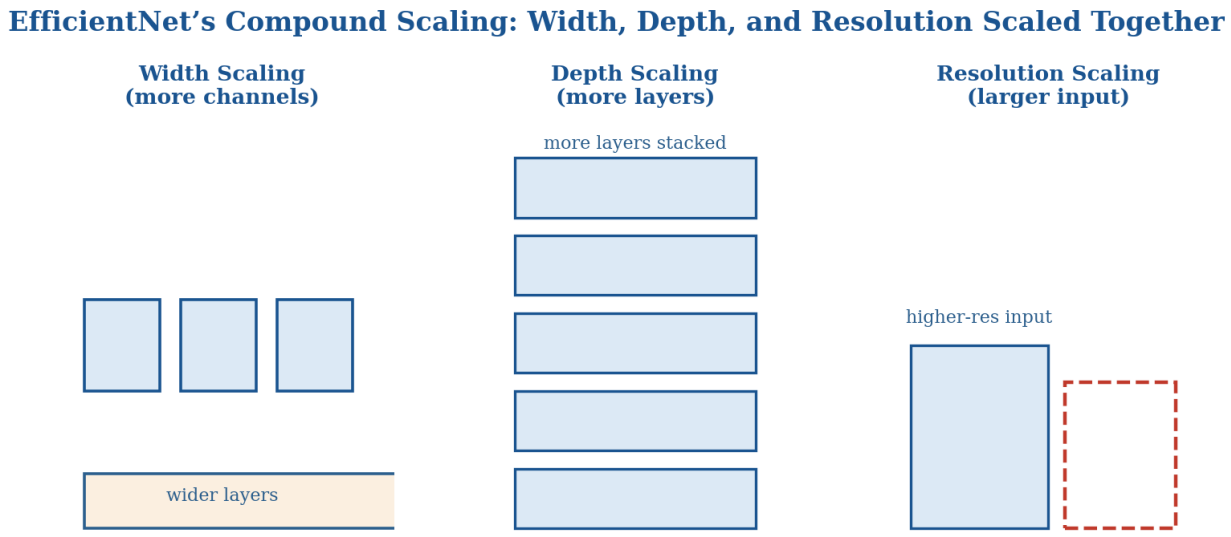

Figure 7. EfficientNet's compound scaling increases network width, depth, and input resolution together in a fixed ratio, rather than scaling any single dimension in isolation.

Worked Example: Intuition for Why Compound Scaling Beats Single-Dimension Scaling

Scaling resolution alone: a higher-resolution input gives the network more spatial detail to work with, but without also increasing depth, the network may lack the receptive field (Day 29 §3.3) needed to integrate that additional detail into larger-scale patterns — the extra resolution is partly wasted.

Scaling depth alone: a deeper network can represent more complex functions, but without wider layers or higher input resolution, each layer may not have enough channel capacity or spatial detail to fully exploit that added depth.

Compound scaling: increasing width, depth, and resolution together ensures each additional unit of compute is matched by a proportional increase along every dimension the network needs to actually use it effectively — the empirical justification behind EfficientNet's consistent accuracy-per-FLOP advantage across its entire B0–B7 family, visible already in Figure 1's timeline.

6.4 Practical Accuracy-vs-Latency Trade-offs

Architecture	Relative Size	Relative Speed	Best Fit
MobileNetV2/V3	Very small	Very fast	Mobile/embedded, tight latency budgets
EfficientNet-B0–B2	Small–moderate	Fast	Balanced accuracy/efficiency on modest hardware
ResNet-50	Moderate	Moderate	General-purpose default pretrained backbone
EfficientNet-B5–B7	Large	Slower	Maximum accuracy where compute allows

This table sets up directly the backbone-selection reasoning Section 9 develops in full, and connects concretely back to Day 26's PP12 assignment: a lightweight face-mask classifier deployed on entry-screening hardware would sit toward this table's top row, while a research pipeline exploring maximum achievable accuracy with server-class GPU resources available would more reasonably sit toward the bottom.

6.5 The MobileNet Lineage: V1 Through V3

MobileNet itself was refined across three successive versions, each addressing a specific limitation identified in its predecessor — the same incremental, problem-driven refinement pattern Day 28 §1.4 identified across the classical feature-detection lineage, recurring here in the efficient-architecture lineage. MobileNetV1 introduced depthwise separable convolutions (Section 6.2) as its central technique, achieving strong efficiency gains but using a comparatively simple, linear stack of these separable convolutions throughout. MobileNetV2 added inverted residual blocks — a structural echo of ResNet's bottleneck design (Section 4.1), but inverted: rather than reducing channels before the expensive spatial convolution as ResNet's bottleneck does, MobileNetV2 first expands channels, applies the depthwise spatial convolution at this expanded width, then projects back down to a narrow "bottleneck" representation between blocks — along with linear bottlenecks (omitting the ReLU activation, Day 29 §5.2, specifically at the narrow bottleneck point, based on the empirical finding that ReLU applied to an already-compressed, low-dimensional representation destroys useful information more readily than at higher-dimensional points in the network). MobileNetV3 combined V2's inverted residual design with Neural Architecture Search (Section 5.4's callout) to automatically tune the exact configuration of each block, plus a new, cheaper approximation to the sigmoid activation (called hard-swish) tuned specifically for efficient mobile hardware execution.

Version	Key Addition	Design Origin
MobileNetV1	Depthwise separable convolutions	Hand-designed
MobileNetV2	Inverted residuals + linear bottlenecks	Hand-designed, ResNet-inspired
MobileNetV3	NAS-tuned blocks + hard-swish activation	Neural Architecture Search

7 Vision Transformers

This section is a conceptual preview only. It exists to answer one specific question — what is a Vision Transformer, at a level sufficient to place it correctly in the architecture landscape Section 9 surveys — and deliberately does not cover patch embedding mechanics, positional encoding details, transformer encoder internals, or training and implementation specifics. Those topics belong to Day 38 (Introduction to Vision Transformers) and the dedicated Vision Transformers module later in this broader research programme, where they receive the same first-principles depth this day has given CNNs.

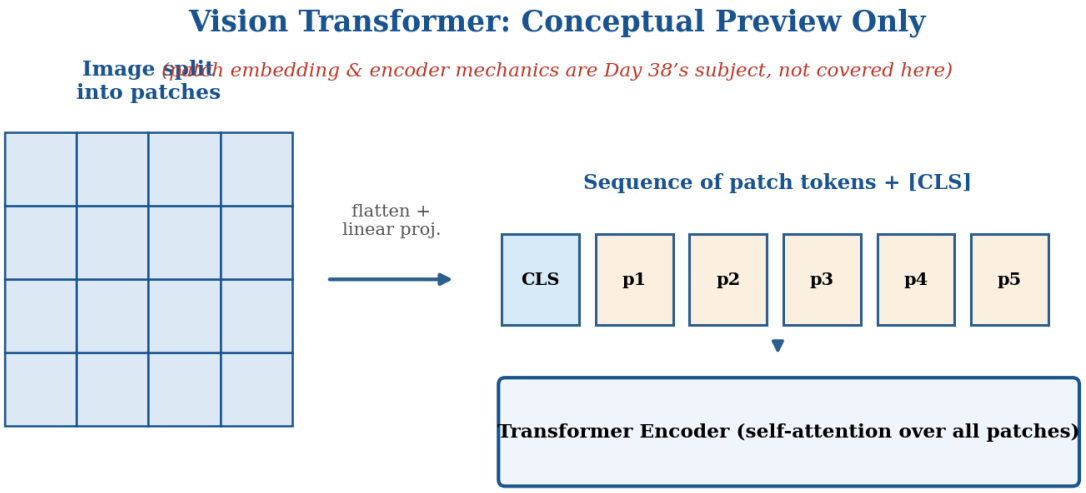

Figure 8. *A conceptual, high-level preview of the Vision Transformer pipeline — patch embedding and transformer encoder mechanics are Day 38's subject, not developed here.*

7.1 What ViT Is, at a High Level

A Vision Transformer (ViT) processes an image by first dividing it into a grid of fixed-size patches (commonly 16×16 pixels), flattening each patch and projecting it into a token embedding — conceptually analogous to how a word is converted into a token embedding in natural language processing. This sequence of patch tokens is then processed by a standard transformer encoder, using the self-attention mechanism to let every patch directly attend to every other patch's content, regardless of spatial distance between them, at every layer of the network.

It is worth contrasting this explicitly against every architecture Sections 2 through 6 covered, since the contrast is what makes ViT a genuinely distinct point in the design space rather than an incremental refinement. Every CNN this day has surveyed — VGG, ResNet, Inception, MobileNet, EfficientNet — processes an image through a hierarchy of local operations: a given unit at a given layer only ever "sees" a receptive field (Day 29 §3.3) that grows gradually with depth, reaching a genuinely global view of the image only in the network's final layers, if at all. A ViT has no such restriction from its very first layer: the self-attention mechanism allows any patch to directly incorporate information from any other patch immediately, with no gradual, depth-dependent receptive-field growth required. This is the structural essence of "replacing convolution with attention" — not a minor implementation detail, but a fundamentally different assumption about how visual information should be allowed to flow through the network.

7.2 The Key Trade-off: Data Hunger vs. Inductive Bias

The single most important conceptual point this preview needs to convey — because Section 9's backbone-selection framework depends on it directly — is the trade-off illustrated in Figure 8. A CNN's local receptive fields and weight sharing (Day 29 §2.4) impose a strong inductive bias: a built-in assumption, baked directly into the architecture, that nearby pixels are more likely to be related than distant ones. This

assumption is usually correct for natural images, and it means a CNN does not need to learn this locality structure from data — it is given for free by the architecture, which is a large part of why CNNs train effectively on comparatively modest labelled datasets. A ViT's self-attention mechanism imposes no such built-in assumption at all: every patch can attend to every other patch equally from the very first layer, meaning any useful locality structure present in real images must be discovered from data rather than assumed by the architecture — which requires substantially more training data before a ViT matches a comparably-sized CNN's performance, but, once enough data is available, allows the network more flexibility to model relationships a CNN's built-in locality bias would never allow it to represent directly.

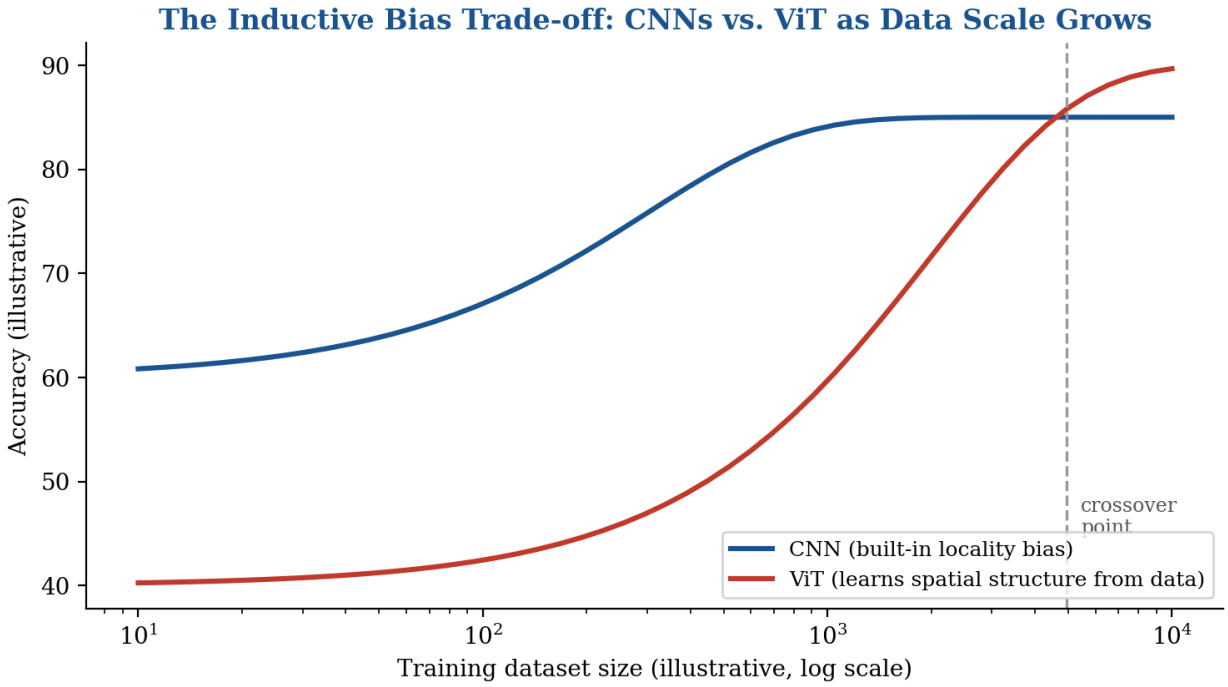

Figure 9. *CNNs outperform ViTs on smaller datasets thanks to their built-in locality bias; ViTs close the gap and can surpass CNNs once enough training data is available to learn spatial structure from scratch.*

7.3 Where ViT Fits in Section 9's Framework

This conceptual understanding — ViT exists, trades inductive bias for flexibility, and needs more data to realise its potential — is exactly what Section 9's backbone-selection framework needs in order to include ViT as one option among several, without requiring the implementation depth Day 38 provides. The deep dive into patch embedding mechanics, positional encoding schemes, and the transformer encoder's internal structure happens in full in Day 38 — this section's job was only to ensure ViT can be recognised, understood at a conceptual level, and correctly positioned in the architecture landscape.

It is worth being explicit about one further consequence of ViT's lack of built-in spatial structure, since it resurfaces directly in Day 38: because self-attention treats the sequence of patch tokens as an unordered set (mathematically, attention has no inherent notion of "position" at all), a ViT must be given explicit positional information — typically a learned positional embedding added to each patch token — or it would have no way to distinguish "this patch is top-left" from "this patch is bottom-right" at all. This positional-

embedding mechanism is one of the specific implementation details this section has deliberately deferred to Day 38, per this section's stated scope boundary, but it is worth knowing it exists: it is the reason ViT can process spatial information at all, despite attention's otherwise position-agnostic mathematical structure.

8 CNN-Transformer Hybrids

8.1 Motivation: Combining Data Efficiency with Global Reasoning

Given Section 7.2's trade-off — CNNs are data-efficient thanks to their built-in locality bias, ViTs are more flexible but data-hungry — a natural research direction combines both approaches within a single architecture: using convolutional layers for their strong local inductive bias and computational efficiency in early processing stages, and attention-based layers for their ability to model long-range, global relationships in later stages, once the network has already extracted reasonably rich local features a CNN is well-suited to provide cheaply.

This hybridisation can happen at several different levels of granularity, worth distinguishing briefly. Some hybrids interleave convolutional and attention layers stage-by-stage, following a CNN-style pyramid structure (Day 29 §6.4) but replacing some later stages' convolutions with attention blocks. Others, like Swin (Section 8.2), redesign the attention mechanism itself to behave more like a convolution — local, windowed, and hierarchical — while remaining, formally, an attention-based architecture throughout. Still others, like ConvNeXt (Section 8.3), take the reverse approach: keeping the architecture purely convolutional throughout, while adopting training recipes, normalisation choices, and design conventions that originated in the transformer literature. All three strategies are legitimate responses to the same underlying motivation, and none has emerged as a definitively superior approach to the others — precisely the "close, context-dependent choice" Section 8.4 concludes with.

8.2 Swin Transformer: A Named Example

Swin Transformer is worth naming as a concrete, influential example of this hybrid design space, though its architectural details — hierarchical windowed attention, in particular — are reserved for the dedicated Advanced Vision Transformer Architectures material later in this broader research programme, which covers it alongside the other advanced ViT variants it is best understood in the context of. At a conceptual level: Swin restricts self-attention computation to local windows (echoing a CNN's locality bias, Section 7.2) rather than ViT's original global-attention-everywhere design, while shifting these windows between successive layers to allow information to eventually flow across window boundaries — producing a hierarchical, multi-resolution feature pyramid more directly comparable to a CNN backbone's stage-by-stage structure (Day 29 §6.4) than ViT's original single-resolution design, and making Swin considerably more practical than plain ViT as a drop-in backbone for the detection and segmentation tasks Day 31 and Day 33 cover.

8.3 ConvNeXt: A 'Modernized CNN'

ConvNeXt takes the opposite direction from Swin's CNN-toward-transformer hybridisation: rather than adding convolutional locality bias into a transformer, ConvNeXt starts from a standard ResNet-style CNN (Section 4) and systematically absorbs transformer-era design choices into it — larger kernel sizes than the small 3×3 kernels VGG and ResNet standardised on, layer normalization in place of batch normalization (Day 29 §8.3), GELU activation (Day 29 §5.3) in place of ReLU, and other training and architectural refinements borrowed from the transformer literature — while remaining a purely convolutional architecture throughout, with no attention mechanism at all. ConvNeXt's results are a useful, concrete counterpoint to any assumption that transformers are unconditionally superior to CNNs: a sufficiently modernised, carefully-tuned CNN can match or exceed a comparably-sized ViT's accuracy, without needing any of ViT's additional training data or attention-specific machinery at all.

Worked Example: What ConvNeXt's Result Actually Demonstrates

The original ConvNeXt paper systematically modernised a ResNet-50 baseline (Section 4.3) one design choice at a time — changing the training recipe first (longer training, stronger augmentation per Day 27 §6), then the macro design (stage compute ratio, "patchify" stem), then ResNeXt-style grouped convolutions (Section 4.4), then inverted bottlenecks (echoing MobileNetV2's design, §6.5), then larger kernel sizes, and finally a series of smaller layer-level changes (GELU, fewer activations, layer normalization).

Each individual change contributed a small accuracy improvement; cumulatively, they closed nearly the entire gap to a comparably-sized Swin Transformer (§8.2), while the network remained a standard convolutional architecture throughout, with zero attention layers added at any point.

The lesson is not that convolution is "secretly better" than attention, but that much of the accuracy gap observed between older CNNs and newer transformers in various benchmarks was attributable to accumulated training-recipe and micro-design improvements developed alongside the transformer literature, rather than to attention itself being an inherently superior mechanism — a genuinely important, nuanced finding for interpreting any CNN-vs-transformer benchmark comparison correctly.

8.4 The Practical Takeaway: A Close, Context-Dependent Choice

The overall lesson from this section, worth carrying forward into Section 9's decision framework, is that the CNN-versus-transformer choice is, in current practice, often a genuinely close call rather than a clear-cut win for either side — and increasingly, the most competitive options (Swin, ConvNeXt) are themselves hybrids or CNN/transformer-adjacent designs that borrow the other lineage's best ideas, rather than "pure" representatives of either original approach. The right choice for a specific project depends far more on the practical constraints Section 9 develops — dataset size, latency budget, deployment target — than on any inherent, universal superiority of one architectural family over the other.

Architecture	Core Mechanism	Attention Used?	Best Fit
Plain ViT (§7)	Global self-attention throughout	Yes, from layer 1	Very large pretraining data available
Swin Transformer	Windowed, shifted attention	Yes, locally then globally	Dense prediction (detection/segmentation)

Architecture	Core Mechanism	Attention Used?	Best Fit
ConvNeXt	Modernised pure convolution	No	Matching ViT accuracy without attention overhead

9 Choosing a Backbone in Practice

This section consolidates every architecture covered across Sections 2 through 8 into a single practical decision framework, illustrated in Figure 9.

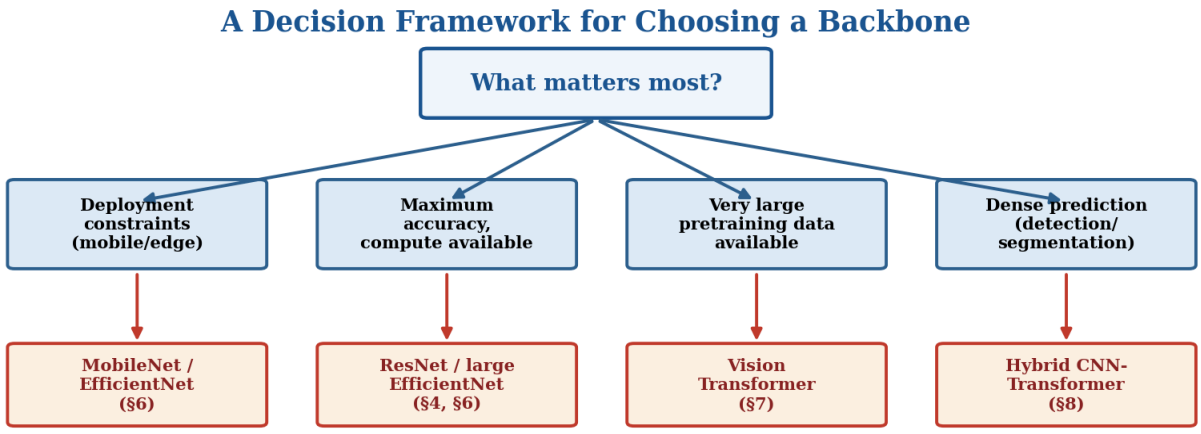

Figure 10. *A practical decision framework for choosing a backbone, synthesising every architecture family this day has covered against the constraint that matters most for a given project.*

Before working through Sections 9.1 through 9.3's individual factors, it is worth stating the framework's overall shape plainly: none of dataset size, latency budget, or fine-tuning strategy should be considered in isolation, since real projects typically face constraints along more than one of these dimensions simultaneously. A project with both a small dataset and a tight latency budget — arguably the most constrained realistic scenario, and one this module's own PP12 assignment approximates — should weight Section 9.2's efficient-architecture guidance most heavily, since Section 6's efficient architectures are, in practice, also reasonably data-efficient by virtue of their smaller parameter count relative to the largest models in Sections 2 through 5, even though data efficiency was not their primary design goal in the way it was for Section 7's inductive-bias discussion.

9.1 Dataset Size

Per Section 7.2's inductive-bias trade-off, dataset size is often the single most decisive factor in the CNN-versus-transformer choice specifically: a project with a small-to-moderate labelled dataset and no access to a very large pretraining corpus should default to a CNN (ResNet, EfficientNet) or a CNN-transformer hybrid (Swin, ConvNeXt) rather than a plain ViT, since a plain ViT's lack of built-in locality bias will very likely underperform a comparably-sized CNN under exactly these data-constrained conditions — directly matching Figure 8's crossover point.

It is worth being specific about what "dataset size" means in this context, since Section 9.3's fine-tuning discussion complicates a naive reading. The relevant quantity is not simply the size of a project's own labelled dataset in isolation, but the effective data the final model has been exposed to across both pretraining and fine-tuning combined. A ViT fine-tuned from a checkpoint pretrained on hundreds of millions of images (as CLIP's image encoder was, per Section 10.3) can perform very well even when fine-tuned on a comparatively small project-specific dataset, since the bulk of the "data hunger" Section 7.2 described has already been satisfied during pretraining, before the project's own small dataset ever enters the picture. The dataset-size guidance in this section applies most directly to training a ViT largely from scratch, or fine-tuning from a comparatively weak or narrowly-scoped pretrained checkpoint — not to fine-tuning from a very large, general-purpose pretrained ViT.

9.2 Latency Budget and Deployment Target

For deployment scenarios with a hard latency or resource budget — mobile, embedded, or real-time applications, directly including this module's own PP12 face-mask-detection scenario from Day 26 — Section 6's efficient architectures (MobileNet, EfficientNet-B0 through B2) are the natural default, per Section 6.4's trade-off table. For server-side deployment with generous compute available and accuracy as the primary objective, a deeper ResNet variant (Section 4.3) or a larger EfficientNet variant remains a strong, well-understood default choice.

9.3 Fine-Tuning vs. Training from Scratch

Per Day 29 §7.5's transfer-learning discussion, the overwhelming majority of practical projects fine-tune a pretrained backbone rather than training from scratch — and the choice of which pretrained backbone to start from should be guided by the same dataset-size and latency considerations above, applied to the pretrained checkpoint being selected rather than to a from-scratch design decision. It is worth noting that pretrained checkpoints are readily available for every architecture family covered in this day (ResNet, EfficientNet, and increasingly ViT and its hybrids), so the practical choice rarely requires implementing an architecture from its original paper — the decision is almost always which existing pretrained checkpoint to start fine-tuning from.

Worked Example: Applying the Full Framework to a Concrete Scenario

Scenario: A research team has 3,000 labelled medical images (a small dataset by deep learning standards) and needs to deploy the resulting classifier on a hospital workstation with a modest GPU, no strict real-time requirement.

Dataset size (§9.1): 3,000 images is small — this rules out a plain ViT trained or fine-tuned without a very strong pretrained starting point (Figure 8's crossover point), pointing toward a CNN or hybrid.

Latency budget (§9.2): no strict real-time requirement and a workstation GPU available rules out needing MobileNet-tier efficiency (§6) — a standard ResNet-50 or mid-sized EfficientNet is comfortably affordable.

Conclusion: a ResNet-50 or EfficientNet-B3-ish pretrained backbone (§4, §6), fine-tuned (§9.3) on the 3,000-image dataset, is the framework's recommended starting point — directly the kind of reasoning Day 37's medical imaging content assumes as background when it revisits backbone choice in that specific high-stakes domain.

9.4 Forward to Day 31: Detection Backbones

This backbone-selection framework applies directly, not merely by analogy, to Day 31's object detection content: every detection architecture Day 31 covers (Faster R-CNN, YOLO, SSD) requires a backbone network to extract features before its detection-specific head operates on them, and the specific backbone choice — a ResNet variant, an EfficientNet variant, or increasingly a Swin-style hybrid for their favourable multi-resolution feature pyramid structure (Section 8.2) — is exactly the choice this section's framework guides. Day 31 §1.1 forward-references this day's backbone content directly for this reason, and Day 33's segmentation architectures depend on the same backbone-selection reasoning once again.

```
import torchvision.models as models

# Section 9's framework in code: swapping backbones is a one-line change

# Tight latency budget (§9.2) -> efficient architecture (§6)
backbone = models.mobilenet_v3_small(weights="IMAGENET1K_V1")

# General-purpose default (§9.3) -> ResNet-50
backbone = models.resnet50(weights="IMAGENET1K_V2")

# Maximum accuracy, compute available -> larger EfficientNet
backbone = models.efficientnet_b5(weights="IMAGENET1K_V1")

# Large pretraining data available, dense prediction tasks -> Swin hybrid
backbone = models.swin_v2_b(weights="IMAGENET1K_V1")
```

Notice that every one of these four calls follows an identical pattern despite representing four architecturally very different networks (Sections 2–8) — a direct practical consequence of Section 9.3's point that the field has converged on a shared, consistent interface for pretrained backbones, meaning the effort in a real project goes into deciding which backbone to select (per this section's framework) rather than into implementing any of them from scratch.

10 Research Frontiers

10.1 Scaling Laws for Vision Models

Research into neural scaling laws — originally developed in the context of large language models — has increasingly been applied to vision architectures as well, studying how accuracy improves as a predictable function of model size, dataset size, and training compute, extending the empirical compound-scaling insight EfficientNet (Section 6.3) discovered through direct experimentation into a more general, quantitatively predictive theory. Understanding these scaling relationships helps predict, before undertaking an expensive training run, roughly how much additional accuracy a given increase in model size or data is likely to yield — directly relevant to planning research and deployment decisions at scale.

A specific, practically important finding from this line of research concerns exactly the CNN-versus-ViT crossover point Section 7.2 and Figure 8 introduced conceptually: scaling laws research has shown that this crossover is not a fixed, universal dataset size, but shifts depending on the specific architecture variants being compared, the pretraining data's diversity (not merely its raw size), and the downstream task. This means Section 9.1's dataset-size guidance should be treated as a useful rule of thumb rather than a precise, universally applicable threshold — the safest practical approach remains empirical validation on a specific project's actual data and task, rather than relying solely on general scaling-law intuition to make the final architecture decision.

10.2 Self-Supervised Pretraining: DINO and MAE

Day 29 §10.4 introduced self-supervised CNN pretraining as a research frontier; the same underlying motivation applies to the architectures this day has covered more broadly, CNN and transformer alike. DINO (previewed already in Day 26 §10.1 and Day 28 §10.4) demonstrates self-supervised pretraining applied specifically to Vision Transformers, producing representations with striking emergent properties — including, as Day 28 §10.4 noted, features useful for correspondence and matching tasks despite no explicit training objective for that capability. MAE (Masked Autoencoders) applies a conceptually different self-supervised objective specifically well-suited to ViT's patch-based structure (Section 7.1): randomly masking a large fraction of an image's patches and training the network to reconstruct the missing content from the remaining visible patches, a vision-specific analogue of the masked-language-modelling objective used to pretrain large language models.

It is worth noting why MAE's masking-based objective is particularly well-suited to ViT's architecture specifically, rather than being equally natural for a CNN. Because ViT already processes an image as a discrete sequence of patch tokens (Section 7.1), masking a subset of those tokens and asking the network to predict their content from the remaining tokens is a completely natural extension of the architecture's existing input representation — the masked tokens can simply be dropped from the input sequence entirely during the encoder's forward pass, substantially reducing MAE's pretraining compute cost relative to processing a full, unmasked image. A CNN's dense, grid-structured feature maps (Day 29 §2) do not offer an equally natural way to "drop" a masked region from computation in the same way, which is part of why MAE-style masked pretraining has been most prominently and effectively applied to ViT-based architectures specifically.

10.3 Multimodal Backbones: CLIP

CLIP, previewed already in Day 26 §10.1 and Day 28 §1.2's vision-language forward references, uses a Vision Transformer (or, in some variants, a CNN) as its image encoder, trained jointly with a text encoder via a contrastive objective linking matching image-text pairs. This positions the architectures this day has covered not merely as standalone classifiers, but as one half of an increasingly important multimodal foundation model paradigm — a preview of the research directions Days 34–35's generative models and the dedicated Vision-Language Models module later in this broader research programme develop in depth.

CLIP's own internal ablation studies, comparing a ResNet-based image encoder against a ViT-based one under otherwise identical training conditions, found the ViT variant achieved better accuracy per unit of training compute at the very large data scale CLIP was trained on (hundreds of millions of image-text pairs) — a direct, large-scale empirical data point supporting Section 7.2's inductive-bias trade-off: at sufficiently large scale, ViT's greater flexibility outweighs the efficiency advantage a CNN's built-in locality bias provides at smaller scale.

10.4 Architecture-Agnostic Foundation Models

A broader trend worth naming, tying together this entire day's survey: the field is increasingly moving toward foundation models where the specific backbone architecture (a particular CNN family, a particular ViT variant, a particular hybrid) matters somewhat less than the scale of pretraining data and compute applied to it, echoing Section 10.1's scaling-law perspective. This does not make the architectural understanding this day has developed obsolete — every foundation model still has to choose some specific backbone architecture, and the trade-offs Section 9's framework catalogued (dataset size, latency, deployment target) remain just as relevant when selecting or designing that backbone — but it does suggest that architecture-specific advantages are becoming smaller relative to the effect of scale, a trend worth watching as this broader research programme continues into its generative modelling and vision-language content.

10.5 Closing Perspective: From Architecture Zoo to Practical Framework

It is worth stepping back, at the close of this day's research-frontiers discussion, to state plainly what this day has actually accomplished. Day 29 built a first-principles understanding of what a single convolutional layer computes and how a basic CNN is trained; this day has traced how that basic building block was elaborated, refined, and eventually complemented by an entirely different mechanism (self-attention) across a full decade of research — VGG's disciplined depth, ResNet's solution to the vanishing-gradient ceiling that depth alone runs into, Inception's multi-scale alternative, MobileNet and EfficientNet's efficiency-first redesign, and ViT's structurally distinct approach, with Section 8's hybrids showing that even this apparent architecture-family divide is increasingly porous. Section 9's decision framework is the payoff this entire lineage has been building toward: not a claim that any single architecture is universally best, but a principled way to match a project's actual constraints — dataset size, latency budget, deployment target — against the specific trade-offs each architectural lineage in this day was designed to make. Day 31, starting immediately after this one, puts this exact framework to immediate practical use: every object detector Day 31 covers needs a backbone, and the choice of which backbone to use is precisely the choice this day has equipped you to make.

11 Common Pitfalls: A Practical Debugging Checklist

Because this day covers architecture selection and comparison rather than a single implementation, the pitfalls here are primarily about experimental rigor and correct interpretation of results, rather than the code-level bugs Days 26–29's checklists focused on. The following checks catch the most common mistakes when comparing or selecting between the architectures this day has covered.

- **Compare architectures at matched parameter or compute budgets, not just accuracy** — per §1.2, a larger, slower network will very often report higher raw accuracy than a smaller one; the meaningful comparison for Section 9's decision framework is accuracy-per-unit-of-compute or accuracy-per-parameter, not accuracy alone.
- **Do not assume a ViT is a drop-in replacement for a CNN backbone without checking dataset size** — per §7.2's crossover point, substituting a plain ViT for a CNN backbone on a small or moderate fine-tuning dataset frequently underperforms the CNN it replaced, a surprising result to anyone unaware of the inductive-bias trade-off.
- **Verify a pretrained checkpoint's expected input preprocessing matches your pipeline** — different architecture families sometimes expect different input normalization statistics or resolutions (Day 27 §5.2); mismatching these silently degrades a pretrained backbone's performance in exactly the way Day 27's forgotten-normalization bug described.
- **When benchmarking latency, measure on the actual target deployment hardware** — per §6.4, relative speed rankings between architectures can shift meaningfully between GPU, CPU, and mobile/embedded hardware, since some architectural techniques (depthwise separable convolutions, §6.2, in particular) benefit far less from certain hardware's parallelism than standard convolutions do.
- **Do not conflate "deeper" with "better" when reading an unfamiliar architecture's reported depth** — per §4.3's diminishing-returns pattern within the ResNet family alone, additional depth's marginal accuracy benefit shrinks well before it reaches zero cost — always check whether a deeper variant's accuracy gain justifies its added compute cost for your specific application.
- **Re-verify the framework's conclusion empirically before committing to it** — per §10.1's scaling-law caveat, Section 9's decision framework provides a strong, well-reasoned starting point, but the crossover points and trade-offs it describes are approximate rules of thumb, not precise guarantees — a quick empirical comparison on a held-out validation set, where feasible, remains the most reliable way to confirm a backbone choice before committing to it for a full project.

11.1 A Note on Fair Architecture Comparison

Comparing two architectures fairly is more subtle than it first appears, and worth a dedicated note given how central comparison is to this entire day's content. A meaningful comparison should control for training recipe (learning rate schedule, augmentation strategy, number of training epochs — Day 29 §7), since a modern, carefully-tuned training recipe applied to an older architecture can close a substantial fraction of the accuracy gap against a newer architecture trained with an older or less-tuned recipe, potentially leading

to the wrong conclusion about which architecture is genuinely better. Published benchmark numbers for different architectures were also not always produced under identical training conditions, data augmentation policies, or even identical definitions of "top-1 accuracy" evaluation protocol — a discrepancy worth being aware of before treating a leaderboard's ranking as a precise, apples-to-apples comparison rather than a useful but imperfect guide.

Glossary of Terms

This day has introduced a substantial amount of specialised vocabulary. The glossary below collects the most important terms in one place for quick reference, with a pointer back to the section where each is developed in full.

Term	Definition (see section for full detail)
Degradation problem	Both train and test accuracy worsen with added depth in a plain network (§3.1)
Residual / skip connection	A direct path carrying a block's input around its layers, added back at the end (§3.3)
Bottleneck block	A 1×1-3×3-1×1 residual block design that keeps compute manageable (§4.1)
Cardinality	The number of parallel branches in a ResNeXt block (§4.4)
Inception module	Parallel convolutions at multiple kernel sizes, concatenated (§5.1)
Factorised convolution	A K×K convolution decomposed into cheaper 1×K and K×1 steps (§5.3)
Depthwise separable convolution	A standard convolution factorised into per-channel + 1×1 stages (§6.2)
Compound scaling	Scaling network width, depth, and resolution together in a fixed ratio (§6.3)
Vision Transformer (ViT)	A transformer applied to a sequence of image patches (§7.1)
Inductive bias	A built-in architectural assumption, such as a CNN's locality (§7.2)
Positional embedding	Learned information telling a ViT where each patch sits spatially (§7.3)
Hybrid architecture	A design combining convolutional and attention-based components (§8.1)
Neural Architecture Search (NAS)	Automated search over a space of possible architectures (§5.4)
Scaling laws	Predictable relationships between accuracy, model size, data, and compute (§10.1)

10.6 Open Questions Heading Into the Next Modules

This day closes the "Computer Vision Fundamentals" backbone-architecture narrative, but several open questions it has raised are picked up directly in the modules that follow this one. Whether hybrid architectures (Section 8) or increasingly large plain ViTs (Section 7) will dominate future large-scale vision systems remains an active, unsettled research question — Days 38 through 42's dedicated Vision Transformers and Vision-Language Models content engages with this question directly, well beyond this

day's conceptual preview. Whether scaling laws (Section 10.1) will continue to hold as predictably at even larger model and data scales than currently studied is likewise an open empirical question with direct implications for how future architecture research allocates its effort between new architectural ideas and simply scaling existing ones further. Keeping these open questions in view is a useful habit heading into the remainder of this broader research programme: the architectures this day has catalogued are a snapshot of a still-actively-evolving field, not a finished, closed body of knowledge.

Chapter Summary: Key Takeaways

- **Every architecture in this day responds to the same depth-width-efficiency tension** (§1.1) — recognising this turns a list of named models into one continuous story.
- **VGG showed that stacked small kernels beat one large kernel** (§2.1) — fewer parameters, more non-linearity, same receptive field.
- **The vanishing gradient problem, not overfitting, limits plain network depth** (§3.1–3.2) — and residual connections (§3.3–3.4) solve it by giving gradients an unimpeded path back to early layers.
- **ResNet and Inception pursued different levers** (§5.4) — depth-through-residuals versus width-through-parallel-branches — both valid answers to §1.1's tension.
- **MobileNet and EfficientNet optimise for compute budget, not just accuracy** (§6) — depthwise separable convolutions and compound scaling are the two central techniques.
- **ViT trades inductive bias for flexibility** (§7.2) — more data-hungry than a CNN, but with more headroom once enough data is available; full mechanics deferred to Day 38.
- **The CNN-vs-transformer choice is often close, and hybrids (Swin, ConvNeXt) increasingly blur the line** (§8.4) — dataset size and deployment constraints matter more than either family's inherent superiority.
- **Backbone selection is a practical decision, not a search for the single "best" architecture** (§9) — and this framework applies directly to Day 31's detection backbones and Day 33's segmentation architectures.

End of Day 30 — ready for gap review and expansion into full compendium.

Theory-only day — no assigned practical project. Ready for gap review and expansion into full compendium.